

Arithmetic / Logic Instructions:

Increment the contents of a register by a value (decimal/binary/hexa-decimal)

ADD AX, 4

Add the contents of a register with the contents of another register

ADD AX, BX

Subtract a value (decimal/binary/hexa-decimal) from the contents of a register

SUB DL, 4

Subtract the contents of a register from the contents of another register

SUB DX, CX

Multiply AX by BX, the result will be in AX

MUL BX

Divide the contents of AX register with the value of CL and store the result in AX

DIV CL

Increase or Decrease the contents of BX register by 1

INC BX ; Increase
DEC BX ; Decrease

Clear the contents of AX register
XOR AX, AX

Negation and NOT of a register value
NEG AL; 2's Complement
NOT AL; 1's Complement

Assembly Language Program

Example:

```
ORG 0100h
MAIN PROC
; display prompt
MOV AH, 2 #output
MOV DL, '?'
INT 21h
; input a character
MOV AH, 1 #input
INT 21h
MOV BL, AL
; go to a new line with carriage return
MOV AH, 2
MOV DL, 0DH #carriage return
INT 21h
MOV DL, 0AH #line feed(new line)
```

```
INT 21h
; display character
MOV DL, BL
INT 21h
; return to DOS
MOV AH, 4CH
INT 21H
MAIN ENDP
END MAIN
RET
```

CONDITIONS

Jump if zero flag ZF=1 **JZ** zero
Jump if zero flag ZF=0 **JNZ** notzero
Jump if greater **JG** greater
Jump if greater than or equal **JGE** notless
Jump if less **JL** less
Jump if less than or equal **JLE** notgreater
Jump if Below **JB** smaller
Jump if carry flag CF=1 **JC** carry

String Display Instruction

At first define the string to be displayed under DATA SEGMENT:

```
.DATA
test_string DB 'My first string', 0Dh, 0Ah, '$'
```

Then, display the string in the command prompt as:

```
MOV AH, 9
LEA DX, test_string
INT 21h
```

Assembly Language Program Example for Loop:

Count-controlled LOOP to display a row of 50 stars (*).

```
org 100h
.DATA ; Data segment starts
.CODE ; Code segment starts
MAIN PROC
mov ax, @DATA
mov ds, ax
xor cx, cx ; reset the CX register
mov cx, 50
mov ah, 2
mov dl, '*'
top: int 21h
loop top
mov ah, 4ch ; equivalent function number of RETURN
int 21h ; Input-Output Interrupt
MAIN ENDP
END MAIN
RET
```

Array

```
d DB 5 DUP(1, 2)
d DB 1, 2, 1, 2, 1, 2, 1, 2, 1, 2
Both same
```

Assembly Language Program Example for Array:

To derive summation of a series 1 + 2 + 3 using array. Here, value of N is given by user where N=3 and output is shown in the output window:

```
org 100h
.DATA ; Data segment starts
A db 3, 1, 2 ;1-D array for number
B db 00h
message db 'Enter the value of N:$' ;1-D array for string
.CODE ; Code segment starts
MAIN PROC
mov ax, @DATA
mov ds, ax
xor ax, ax
mov si, OFFSET A
mov di, OFFSET B
mov dx, OFFSET message ; Load Effective Address of the message in DX register
; lea dx, message ; (similar meaning like Load Effective Address)
mov ah, 09h ;display string function
int 21h ;display message
mov ah, 01h
int 21h
mov cl, al
sub cl, 48 ; to convert the ascii value of 3 to decimal 3
xor al, al
Loop_1:
add al, [Si]
inc Si
loop Loop_1
mov bl, al
add bl, 48 ; to convert the ascii value of the output to decimal
mov ah, 02h
mov dl, bl
int 21h
MAIN ENDP
END MAIN
RET
```

Assembly Language Program Example for Procedure:

```
ORG 0100h
.DATA
StrArray DB 'Hello World!!$' ; .CODE
MAIN PROC
MOV AX, @DATA
MOV DS, AX
LEA DX, StrArray ; set DX to point to 1
st element of string array StrArray
```

```
CALL USER ; call procedure
MOV AH, 4Ch
MOV AL, 00h ; a code after procedure
call and return
INT 21h ; exit to DOS
MAIN ENDP
USER PROC ; declare a procedure
named USER
MOV AH, 09h
INT 21h
RET ; return to MAIN procedure
USER ENDP ; end of procedure USER
END MAIN ; end of program
```

MACRO

Assembly Language Program Example for Macro and Procedure:
MDSPLY_STRING MACRO STRING ;
Declaration of MACRO

```
MOV DX, OFFSET STRING
CALL DSPLY_STRING ; Calling the
Procedure
```

```
ENDM
ORG 0100h
.DATA
MESSAGE1 DB 'Microprocessors and',
0DH, 0AH, '$'
MESSAGE2 DB 'Assembly Language$'
.CODE
MAIN PROC
MOV AX, @DATA
MOV DS, AX
MDSPLY_STRING MESSAGE1 ; 1st Call
of the MACRO
MDSPLY_STRING MESSAGE2 ; 2nd
Call of the MACRO
MOV AH, 4CH ; Return to DOS
INT 21h
MAIN ENDP
DSPLY_STRING PROC ; Declaration of
PROCEDURE
MOV AH, 09h
INT 21h
RET
DSPLY_STRING ENDP
END MAIN
```

LAB 1

```
(30 + 15) * (575 - 225) + 210
MOV AX, 575 ; Load 575 into AX
SUB AX, 225 ; AX = 350
MOV BX, 30 ; Load 30 into BX
ADD BX, 15 ; BX = 45
MUL BX ; DX:AX = AX * BX (350 *
45 = 15750)
ADD AX, 210 ; AX = 15750 + 210 =
15960
```

```
F = (C* 10 / 5) + 32 - 1
.DATA
F DW ? ; Define variable F
.CODE
MOV AX, 260 ; Load Celsius value
MOV BX, 10 ; Multiplier
MUL BX ; DX:AX = 2600
MOV BX, 5 ; Divisor
DIV BX ; AX = 2600 / 5 = 520
ADD AX, 32 ; AX = 552
SUB AX, 1 ; AX = 551
MOV F, AX ; Store result in variable
```

LAB 2

Even or Odd ASCII Checker

```
ORG 0100h
.DATA
MSG_IN DB 'Input: $'
MSG_OUT DB 0DH, 0AH, 'Output: $'
```

```
.CODE
MAIN PROC
MOV AX, @DATA
MOV DS, AX

; Display Input Prompt
MOV AH, 09h
LEA DX, MSG_IN
INT 21h

; Input a character
MOV AH, 01h
INT 21h
MOV BL, AL ; Store character in BL

; Display Output Prompt
MOV AH, 09h
LEA DX, MSG_OUT
INT 21h

; Check if Even or Odd
; Even numbers have 0 in the last bit,
Odd have 1
TEST BL, 1
JZ IS_EVEN ; (Last bit was 0)
```

```
IS_ODD:
MOV DL, 'O'
JMP DISPLAY
```

```
IS_EVEN:
MOV DL, 'E'
```

```
DISPLAY:
MOV AH, 02h
INT 21h
```

```
; Return to DOS
MOV AH, 4CH
INT 21H
MAIN ENDP
END MAIN
```

Next and Previous 5 Letters (Case Toggle)

```
ORG 0100h
.DATA
MSG_PROMPT DB 'Input: $'
NEWLINE DB 0DH, 0AH, '$'
```

```
.CODE
MAIN PROC
MOV AX, @DATA
MOV DS, AX

; Prompt Input
MOV AH, 09h
LEA DX, MSG_PROMPT
INT 21h

; Get Character
MOV AH, 01h
INT 21h
MOV BL, AL ; BL = Input char

; Toggle Case Logic
CMP BL, 'Z'
JLE IS_UPPER ; If Input <= 'Z', Upper
; Was Lowercase -> Convert to Upper
for output
SUB BL, 32
MOV BH, 1 ; Flag: 1 means input
was lower
JMP START_GEN

IS_UPPER:
ADD BL, 32 ; Was Upper -> Convert
to Lower for output
MOV BH, 0 ; Flag: 0 means input
was upper

START_GEN:
; New Line for Next 5
MOV AH, 09h
LEA DX, NEWLINE
INT 21h
```

```

; NEXT 5 Letters
MOV CL, 5
MOV AL, BL ; Use AL for temporary
math
NEXT_LOOP:
    INC AL
    MOV DL, AL
    MOV AH, 02h
    INT 21h
    LOOP NEXT_LOOP

```

```

; New Line for Previous 5
MOV AH, 09h
LEA DX, NEWLINE
INT 21h

```

```

; PREVIOUS 5 Letters
MOV CL, 5
MOV AL, BL ; Reset AL to base
case-toggled input
PREV_LOOP:
    DEC AL
    MOV DL, AL
    MOV AH, 02h
    INT 21h
    LOOP PREV_LOOP

```

```

; Return to DOS
MOV AH, 4CH
INT 21H
MAIN ENDP
END MAIN

```

Case Converter

```

ORG 0100h
.DATA
MSG1 DB 'Input: $'
MSG2 DB 0DH, 0AH, 'Output: $'

```

```

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

```

```

; Display Input Prompt
MOV AH, 09h
LEA DX, MSG1
INT 21h

```

```

; Input a character
MOV AH, 01h
INT 21h
MOV BL, AL ; Store input in BL

```

```

; Display Output Prompt
MOV AH, 09h
LEA DX, MSG2
INT 21h

```

```

; Logic: Check if Uppercase or
Lowercase
    CMP BL, 'Z' ; Compare input with
'Z' (90)
    JLE CONVERT_TO_LOWER ; If BL <=
'Z', it's Uppercase

```

```

CONVERT_TO_UPPER:
    SUB BL, 32 ; Convert lowercase
to uppercase
    JMP DISPLAY

```

```

CONVERT_TO_LOWER:
    ADD BL, 32 ; Convert uppercase
to lowercase

```

```

DISPLAY:
    MOV AH, 02h
    MOV DL, BL ; Move modified
character to DL
    INT 21h ; Print character

```

```

; Return to DOS
MOV AH, 4CH
INT 21H
MAIN ENDP
END MAIN
RET

```

Character Frequency Counter

```

ORG 0100h
.DATA
input_string DB 'We are IUT Students',
0Dh, 0Ah, '$'
prompt_char DB 0Dh, 0Ah, 'Input: $'
prompt_out DB 0Dh, 0Ah, 'Output: $'
count DB 0 ; Variable to
store frequency

```

```

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

```

```

; 1. Display the stored string
MOV AH, 09h
LEA DX, input_string
INT 21h

```

```

; 2. Display 'Input: ' prompt
MOV AH, 09h
LEA DX, prompt_char
INT 21h

```

```

; 3. Input a single character from user
MOV AH, 01h
INT 21h
MOV BL, AL ; BL now holds
the character to search for

```

```

; 4. Calculate Frequency using Loop
MOV SI, OFFSET input_string ; SI
points to start of string
XOR CX, CX ; Clear CX
MOV CL, 19 ; Set loop count
to length of string

```

```

COUNT_LOOP:
    MOV AL, [SI] ; Move current
string char to AL
    CMP AL, BL ; Compare
string char with user input
    JNE SKIP_INC ; If not equal,
skip increment
    INC count ; If equal,
increment count variable

```

```

SKIP_INC:
    INC SI ; Move to next
character in string
    LOOP COUNT_LOOP

```

```

; 5. Display 'Output: ' prompt
MOV AH, 09h
LEA DX, prompt_out
INT 21h

```

```

; 6. Display the result (Frequency)
MOV DL, count
ADD DL, 48 ; Convert
numeric value to ASCII ('0' = 48)
MOV AH, 02h
INT 21h

```

```

; Return to DOS
MOV AH, 4CH
INT 21H
MAIN ENDP
END MAIN
RET

```

String Length Counter

```

ORG 0100h
.DATA
; Define the string with Carriage Return
(0Dh) and Line Feed (0Ah)
in_string DB 'IUT is an International
University', 0Dh, 0Ah, '$'

```

```

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

```

```

; 1. Display the string
MOV AH, 09h
LEA DX, in_string
INT 21h

```

```

; 2. Initialize registers for counting

```

```

    XOR DX, DX      ; Clear DX (DL will
be our counter)
    MOV SI, OFFSET in_string

```

COUNT_LOOP:

```

    MOV AL, [SI]    ; Load current
character into AL

```

```

    ; Check for termination characters
    CMP AL, '$'     ; Is it the end of the
string?
    JE FINISH       ; If yes, jump to finish

```

```

    ; Optional: Skip counting CR (0Dh) and
LF (0Ah) if desired
    ; For this task, we count every byte
until '$'

```

```

    INC DL          ; Increment the
character count in DL
    INC SI          ; Move to the next
memory address
    JMP COUNT_LOOP ; Repeat the
process

```

FINISH:

```

    ; The result (34 decimal / 22h) is now
stored in DL.

```

```

    ; Return to DOS
    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
RET

```

Sequence Summation

```

ORG 0100h
.DATA
    PROMPT DB 'Enter N (1-9): $'
    MSG_ODD DB 0DH, 0AH,
'ODD_SUM: $'
    MSG_EVEN DB 0DH, 0AH,
'EVEN_SUM: $'

    ; Arrays to hold values (max 9
elements)
    ODD_ARR DB 1, 3, 5, 7, 9, 11, 13,
15, 17
    EVEN_ARR DB 2, 4, 6, 8, 10, 12, 14,
16, 18

    ODD_SUM DW 0
    EVEN_SUM DW 0
    N DB 0

```

```

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

```

```

    ; 1. Input N
    MOV AH, 09h
    LEA DX, PROMPT
    INT 21h

    MOV AH, 01h
    INT 21h
    SUB AL, 48 ; Convert ASCII to Decimal
    MOV N, AL ; 2. Calculate ODD_SUM
    XOR AX, AX ; Clear AX for sum
    XOR CX, CX
    MOV CL, N; Loop N times
    MOV SI, OFFSET ODD_ARR

```

```

LOOP_ODD:
    XOR BX, BX
    MOV BL, [SI]
    ADD AX, BX      ; Add current odd
number to sum
    INC SI
    LOOP LOOP_ODD
    MOV ODD_SUM, AX ; Store in
variable

```

```

    ; 3. Calculate EVEN_SUM
    XOR AX, AX
    XOR CX, CX
    MOV CL, N
    DEC CL          ; Example logic
adjustment for N=5, sum=20
    MOV SI, OFFSET EVEN_ARR

```

```

LOOP_EVEN:
    XOR BX, BX
    MOV BL, [SI]
    ADD AX, BX
    INC SI
    LOOP LOOP_EVEN
    MOV EVEN_SUM, AX ; Store in
variable

```

```

    ; 4. Display ODD_SUM
    MOV AH, 09h
    LEA DX, MSG_ODD
    INT 21h
    MOV AX, ODD_SUM
    CALL PRINT_NUM ; Call procedure
to print multi-digit number

```

```

    ; 5. Display EVEN_SUM
    MOV AH, 09h
    LEA DX, MSG_EVEN
    INT 21h
    MOV AX, EVEN_SUM
    CALL PRINT_NUM

```

```

    ; Return to DOS
    MOV AH, 4CH
    INT 21H
MAIN ENDP

```

; --- Procedure to display multi-digit numbers ---

```

PRINT_NUM PROC
    MOV CX, 0
    MOV BX, 10
DIVIDE:
    XOR DX, DX
    DIV BX
    PUSH DX      ; Push remainder to
stack
    INC CX
    CMP AX, 0
    JNE DIVIDE
DISPLAY_NUM:
    POP DX
    ADD DL, 48   ; Convert back to
ASCII
    MOV AH, 02h
    INT 21h
    LOOP DISPLAY_NUM
    RET
PRINT_NUM ENDP

```

END MAIN

Sum of Squares

```

ORG 0100h
.DATA
    PROMPT DB 'Enter the value of N
(2-9): $'
    MSG_RES DB 0DH, 0AH, 'The result
is: $'

    ; Array storing squares: 1^2, 2^2, 3^2,
4^2, 5^2, 6^2, 7^2, 8^2, 9^2
    ; Decimal values: 1, 4, 9, 16, 25, 36,
49, 64, 81
    SQR_ARR DB 1, 4, 9, 16, 25, 36, 49,
64, 81

```

```

    RESULT DW 0
    N_VAL DB 0

```

```

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

```

```

    ; 1. Display Prompt
    MOV AH, 09h
    LEA DX, PROMPT
    INT 21h

    ; 2. Input N as ASCII and adjust
    MOV AH, 01h
    INT 21h
    SUB AL, 48      ; Convert ASCII '5' to
decimal 5
    MOV N_VAL, AL

```

```

; 3. Calculate Sum using Loop and
Array
XOR AX, AX      ; Clear AX to act as
accumulator
XOR CX, CX
MOV CL, N_VAL   ; Set loop counter
to N
LEA SI, SQR_ARR ; Load address
of the squares array

```

```

SUM_LOOP:
XOR BX, BX
MOV BL, [SI]    ; Get square value
from array
ADD AX, BX      ; Add to total
INC SI         ; Move to next element
in array
LOOP SUM_LOOP

```

```

MOV RESULT, AX  ; Store final sum
in RESULT

```

```

; 4. Display Result Message
MOV AH, 09h
LEA DX, MSG_RES
INT 21h

```

```

; 5. Show the numeric output
MOV AX, RESULT
CALL PRINT_NUM

```

```

; Return to DOS
MOV AH, 4CH
INT 21H
MAIN ENDP

```

```

; --- Procedure to print multi-digit numbers
(Decimal) ---
PRINT_NUM PROC
MOV CX, 0
MOV BX, 10
DIV_LOOP:
XOR DX, DX
DIV BX      ; AX / 10
PUSH DX     ; Push remainder
(digit) to stack
INC CX      ; Count digits
CMP AX, 0
JNE DIV_LOOP

```

```

DISP_LOOP:
POP DX      ; Pop digits in correct
order
ADD DL, 48  ; Convert to ASCII
MOV AH, 02h
INT 21h
LOOP DISP_LOOP
RET
PRINT_NUM ENDP

```

```

END MAIN

```

```

RET

```

Average, Largest, and Smallest

```

ORG 0100h
.DATA
PROMPT DB 'Input 5 digits: $'
AVG_MSG DB 0DH, 0AH, 'AVERAGE
= $'
MAX_MSG DB ' LARGEST = $'
MIN_MSG DB ' SMALLEST = $'

NUM_ARRAY DB 5 DUP(?)
AVERAGE DB ?
LARGEST DB ?
SMALLEST DB ?

```

```

.CODE
MAIN PROC
MOV AX, @DATA
MOV DS, AX

```

```

; 1. Display Prompt
MOV AH, 09h
LEA DX, PROMPT
INT 21h

```

```

; 2. Input 5 digits and store in array
MOV CX, 5
MOV SI, 0

```

```

INPUT_LOOP:
MOV AH, 01h
INT 21h
SUB AL, 48      ; Convert ASCII to
Decimal
MOV NUM_ARRAY[SI], AL
INC SI
LOOP INPUT_LOOP

```

```

; 3. Call Procedures
CALL FIND_AVG
CALL FIND_MAX
CALL FIND_MIN

```

```

; 4. Display Results
MOV AH, 09h
LEA DX, AVG_MSG
INT 21h
MOV DL, AVERAGE
ADD DL, 48      ; Convert back to
ASCII

```

```

MOV AH, 02h
INT 21h

MOV AH, 09h
LEA DX, MAX_MSG
INT 21h
MOV DL, LARGEST
ADD DL, 48
MOV AH, 02h
INT 21h

```

```

MOV AH, 09h
LEA DX, MIN_MSG
INT 21h
MOV DL, SMALLEST
ADD DL, 48
MOV AH, 02h
INT 21h

```

```

; Return to DOS
MOV AH, 4CH
INT 21H
MAIN ENDP

```

```

; --- Procedure: Calculate Average ---
FIND_AVG PROC
XOR AX, AX
MOV CX, 5
MOV SI, 0
SUM_LOOP:
ADD AL, NUM_ARRAY[SI]
INC SI
LOOP SUM_LOOP

```

```

MOV BL, 5
DIV BL      ; AX / 5 -> AL is
quotient
MOV AVERAGE, AL
RET
FIND_AVG ENDP

```

```

; --- Procedure: Find Largest ---
FIND_MAX PROC
MOV SI, 0
MOV AL, NUM_ARRAY[SI] ; Assume
first is largest
MOV CX, 4
CHECK_MAX:
INC SI
CMP NUM_ARRAY[SI], AL
JLE SKIP_MAX
MOV AL, NUM_ARRAY[SI]
SKIP_MAX:
LOOP CHECK_MAX
MOV LARGEST, AL
RET
FIND_MAX ENDP

```

```

; --- Procedure: Find Smallest ---
FIND_MIN PROC
MOV SI, 0
MOV AL, NUM_ARRAY[SI] ; Assume
first is smallest
MOV CX, 4
CHECK_MIN:
INC SI
CMP NUM_ARRAY[SI], AL
JGE SKIP_MIN
MOV AL, NUM_ARRAY[SI]
SKIP_MIN:
LOOP CHECK_MIN

```

```

    MOV SMALLEST, AL
    RET
FIND_MIN ENDP

```

END MAIN

GCD_LCM

ORG 0100h

.DATA

```

    PROMPT DB 'Input 2 digits: $'
    MSG_GCD DB 0DH, 0AH, 'GCD = $'
    MSG_LCM DB ' and LCM = $'

```

```

    NUM1 DB ?
    NUM2 DB ?
    GCD DB ?
    LCM DB ?

```

.CODE

MAIN PROC

```

    MOV AX, @DATA
    MOV DS, AX

```

```

; 1. Display Prompt
MOV AH, 09h
LEA DX, PROMPT
INT 21h

```

```

; 2. Input first digit
MOV AH, 01h
INT 21h
SUB AL, 48
MOV NUM1, AL

```

```

; Input space (to match sample "2 4")
MOV AH, 01h
INT 21h

```

```

; Input second digit
MOV AH, 01h
INT 21h
SUB AL, 48
MOV NUM2, AL

```

```

; 3. Call Procedures
CALL FIND_GCD
CALL FIND_LCM

```

```

; 4. Display Results
MOV AH, 09h
LEA DX, MSG_GCD
INT 21h
MOV DL, GCD
ADD DL, 48
MOV AH, 02h
INT 21h

```

```

MOV AH, 09h
LEA DX, MSG_LCM
INT 21h

```

```

    MOV DL, LCM
    ADD DL, 48
    MOV AH, 02h
    INT 21h

```

```

; Return to DOS
MOV AH, 4CH
INT 21H

```

MAIN ENDP

```

; --- Procedure: Find GCD using
Euclidean Algorithm ---

```

FIND_GCD PROC

```

    MOV AL, NUM1
    MOV BL, NUM2

```

GCD_LOOP:

```

    CMP BL, 0
    JE GCD_DONE
    XOR AH, AH ; Clear AH for division
    DIV BL ; AL / BL -> AH is
remainder

```

```

    MOV AL, BL ; Move BL to AL
    MOV BL, AH ; Move remainder to BL
    JMP GCD_LOOP

```

GCD_DONE:

```

    MOV GCD, AL
    RET

```

FIND_GCD ENDP

```

; --- Procedure: Find LCM ---

```

FIND_LCM PROC

```

    MOV AL, NUM1
    MOV BL, NUM2
    MUL BL ; AX = NUM1 * NUM2

```

```

    XOR BH, BH
    MOV BL, GCD ; Divide product by
GCD

```

```

    DIV BL ; AL = (NUM1 * NUM2) /
GCD

```

```

    MOV LCM, AL
    RET

```

FIND_LCM ENDP

END MAIN

Odd/Even Array Separator

```

; Macro definition to pass the array
address and call the logic procedure
MACARRAY MACRO ADDR
    LEA SI, ADDR ; Pass the address
of the array to SI
    CALL SEPARATE_PROC ; Call the
procedure to process
ENDM

```

ORG 0100H

.DATA

```

    ARRAY DB 6 DUP(?)
    PROMPT DB 'Input 6 digits (0-9): $'
    ODD_MSG DB 0DH, 0AH, 'ODD

```

Digits: \$'

```

    EVEN_MSG DB 0DH, 0AH, 'EVEN

```

Digits: \$'

```

    SPACE DB ' $'

```

.CODE

MAIN PROC

```

    MOV AX, @DATA
    MOV DS, AX

```

```

; 1. Display Prompt
MOV AH, 09H
LEA DX, PROMPT
INT 21H

```

```

; 2. Input 6 digits into the array

```

```

    MOV CX, 6
    MOV SI, 0

```

INPUT_LOOP:

```

    MOV AH, 01H
    INT 21H
    SUB AL, 48 ; Convert ASCII to
Decimal

```

```

    MOV ARRAY[SI], AL
    INC SI

```

```

; Optional: skip space/comma in input if
typing like "2, 0, 4..."

```

```

    LOOP INPUT_LOOP

```

```

; 3. Use MACRO to process the array
MACARRAY ARRAY

```

```

; 4. Return to DOS

```

```

    MOV AH, 4CH
    INT 21H

```

MAIN ENDP

```

; --- Procedure to separate and display
ODD and EVEN digits ---

```

SEPARATE_PROC PROC

```

; --- Part A: Display ODD Digits ---

```

```

    MOV AH, 09H
    LEA DX, ODD_MSG
    INT 21H

```

```

    MOV CX, 6
    MOV SI, 0

```

ODD_LOOP:

```

    MOV AL, ARRAY[SI]
    TEST AL, 1 ; Check if bit 0 is set
(Odd)

```

```

    JZ NEXT_ODD ; If Zero flag set,
it's Even, so skip

```

```

; Display the Odd Digit

```

```

    MOV DL, ARRAY[SI]
    ADD DL, 48
    MOV AH, 02H

```

```

INT 21H

; Print space
MOV AH, 09H
LEA DX, SPACE
INT 21H

NEXT_ODD:
INC SI
LOOP ODD_LOOP

; --- Part B: Display EVEN Digits ---
MOV AH, 09H
LEA DX, EVEN_MSG
INT 21H

MOV CX, 6
MOV SI, 0
EVEN_LOOP:
MOV AL, ARRAY[SI]
TEST AL, 1 ; Check if bit 0 is set
JNZ NEXT_EVEN ; If Not Zero, it's
Odd, so skip

; Display the Even Digit
MOV DL, ARRAY[SI]
ADD DL, 48
MOV AH, 02H
INT 21H

; Print space
MOV AH, 09H
LEA DX, SPACE
INT 21H

NEXT_EVEN:
INC SI
LOOP EVEN_LOOP

RET
SEPARATE_PROC ENDP

END MAIN

```

Array Search with Macro and Procedure

```

; Macro to pass array address and target
digit to the procedure
MACARRAY MACRO ADDR, TARGET
LEA SI, ADDR ; Load address of
the array into SI
MOV AL, TARGET ; Move the digit
to be searched into AL
CALL SEARCH_PROC ; Call the
search procedure
ENDM

```

```

ORG 0100h
.DATA
ARRAY DB 2, 0, 4, 7, 1, 9

```

```

PROMPT DB 'Input: $'
FOUND_MSG DB 0DH, 0AH, 'Output:
Digit $'
TEXT_F DB ' Found$'
TEXT_NF DB ' Not Found$'
INPUT_VAL DB ?

.CODE
MAIN PROC
MOV AX, @DATA
MOV DS, AX

; 1. Display Prompt
MOV AH, 09h
LEA DX, PROMPT
INT 21h

; 2. Input the digit to search
MOV AH, 01h
INT 21h
SUB AL, 48 ; Convert ASCII input
to decimal
MOV INPUT_VAL, AL

; 3. Use MACRO to perform search
; We pass the ARRAY address and the
INPUT_VAL
MACARRAY ARRAY, INPUT_VAL

; 4. Return to DOS
MOV AH, 4CH
INT 21H
MAIN ENDP

```

```

; --- Procedure to search for the digit in
the array ---
SEARCH_PROC PROC
MOV CX, 6 ; Array size is 6

SEARCH_LOOP:
CMP [SI], AL ; Compare current
array element with target (AL)
JE DIGIT_FOUND ; If equal, jump
to found section
INC SI ; Move to next element
LOOP SEARCH_LOOP

```

```

; If loop finishes without JE, digit is not
found

```

```

DIGIT_NOT_FOUND:
MOV AH, 09h
LEA DX, FOUND_MSG
INT 21h

```

```

; Display the searched digit
MOV DL, AL
ADD DL, 48
MOV AH, 02h
INT 21h

```

```

MOV AH, 09h

```

```

LEA DX, TEXT_NF
INT 21h
RET

DIGIT_FOUND:
MOV AH, 09h
LEA DX, FOUND_MSG
INT 21h

; Display the found digit
MOV DL, AL
ADD DL, 48
MOV AH, 02h
INT 21h

MOV AH, 09h
LEA DX, TEXT_F
INT 21h
RET
SEARCH_PROC ENDP

END MAIN

```

Task 0: N-th Character from Input

```

ORG 0100h
.CODE
MAIN PROC
    MOV AH, 1      ; Input Alpha
    INT 21h
    MOV BL, AL     ; Store Alpha in BL

    MOV AH, 1      ; Input N
    INT 21h
    SUB AL, 48     ; ASCII to Decimal

    ADD BL, AL     ; BL = Alpha + N

    MOV AH, 2
    MOV DL, 0Dh   ; Newline
    INT 21h
    MOV DL, 0Ah
    INT 21h

    MOV DL, BL     ; Display result
    INT 21h

    MOV AH, 4Ch
    INT 21h
MAIN ENDP
END MAIN

```

Tasks 1 & 2: Reverse String with Case Toggle

```

; Task 1 Example (Lower to Upper Reverse)
ORG 0100h
.DATA
    ARR DB 6 DUP(?) ; Example for 6 letters
.CODE
MAIN PROC
    MOV CX, 6
    MOV SI, 0
INPUT:
    MOV AH, 1
    INT 21h
    SUB AL, 32    ; Toggle to Uppercase immediately
    PUSH AX      ; Push onto stack
    LOOP INPUT

    MOV AH, 2
    MOV DL, 0Dh
    INT 21h
    MOV DL, 0Ah
    INT 21h

    MOV CX, 6
OUTPUT:
    POP DX       ; Pop reverses the order
    MOV AH, 2
    INT 21h
    LOOP OUTPUT

```

```

    MOV AH, 4Ch
    INT 21h
MAIN ENDP
END MAIN

```

Task 3 & 4: Even/Odd Digits up to N

```

ORG 0100h
.CODE
MAIN PROC
    MOV AH, 1
    INT 21h
    SUB AL, 48    ; N in AL
    MOV CL, AL
    MOV BL, 0     ; Start from 0

START:
    TEST BL, 1    ; Check if Odd/Even
    JNZ SKIP
    MOV AH, 2
    MOV DL, BL
    ADD DL, 48    ; Decimal to ASCII
    INT 21h
    SKIP:
    INC BL
    CMP BL, CL
    JBE START     ; Loop until BL > N

```

```

    MOV AH, 4Ch
    INT 21h
MAIN ENDP
END MAIN

```

Task 5, 6 & 15, 16: Ascending Sort (Example for 8 Digits)

```

ORG 0100h
.DATA
    ARR DB 8 DUP(?)
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX
    MOV CX, 8
    MOV SI, 0
INP: MOV AH, 1 | INT 21h | SUB AL, 48 |
    MOV ARR[SI], AL | INC SI | LOOP INP

    MOV CX, 8    ; Outer Loop
OUTER:
    PUSH CX
    MOV SI, 0
    MOV CX, 7    ; Inner Loop
INNER:
    MOV AL, ARR[SI]
    CMP AL, ARR[SI+1]
    JL NEXT      ; For Descending,
change JL to JG
    XCHG AL, ARR[SI+1]
    MOV ARR[SI], AL
NEXT:
    INC SI
    LOOP INNER

```

```

POP CX
LOOP OUTER

```

```

; Output Loop (Display sorted ARR)
; ... (Standard AH=2 loop)
MAIN ENDP

```

7. Odd and Even Digit Separator (5 Digits)

```

ORG 100H
.DATA
    INPUT_ARR DB 5 DUP(?)
    MSG1 DB 'Input 5 digits: $'
    MSG_O DB 0DH, 0AH, 'Odd: $'
    MSG_E DB 0DH, 0AH, 'Even: $'
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    ; Display Prompt
    MOV AH, 09H
    LEA DX, MSG1
    INT 21H

    ; Input 5 digits into array
    MOV CX, 5
    MOV SI, 0
GET_INPUT:
    MOV AH, 01H
    INT 21H
    MOV INPUT_ARR[SI], AL
    INC SI
    LOOP GET_INPUT

    ; Output Odds
    MOV AH, 09H
    LEA DX, MSG_O
    INT 21H
    MOV CX, 5
    MOV SI, 0
PRINT_ODD:
    MOV AL, INPUT_ARR[SI]
    TEST AL, 1    ; Check if bit 0 is 1 (Odd)
    JZ NEXT_O     ; If zero flag set, it's even, so skip
    MOV DL, AL
    MOV AH, 02H
    INT 21H
NEXT_O:
    INC SI
    LOOP PRINT_ODD

    ; Output Evens
    MOV AH, 09H
    LEA DX, MSG_E
    INT 21H
    MOV CX, 5
    MOV SI, 0
PRINT_EVEN:

```



```

MOV AL, INPUT_ARR[SI]
TEST AL, 1 ; Check if bit 0 is 1
JNZ NEXT_E ; If Not Zero (is 1),
it's odd, so skip
MOV DL, AL
MOV AH, 02H
INT 21H
NEXT_E:
INC SI
LOOP PRINT_EVEN

MOV AH, 4CH
INT 21H
MAIN ENDP
END MAIN

```

```

8 (Odd Asc) TEST AL, 1 -> JZ
SKIP CMP AL, [SI+1] -> JG SWAP
9 (Odd Desc) TEST AL, 1 -> JZ
SKIP CMP AL, [SI+1] -> JL SWAP
10 (Even Asc) TEST AL, 1 -> JNZ
SKIP CMP AL, [SI+1] -> JG SWAP
11 (Even Desc) TEST AL, 1 -> JNZ
SKIP CMP AL, [SI+1] -> JL SWAP

```

Task 17: Greatest Common Divisor (GCD)

```

FIND_GCD PROC
; AL = Input1, BL = Input2
GCD_L:
CMP BL, 0
JE DONE
XOR AH, AH
DIV BL
MOV AL, BL
MOV BL, AH
JMP GCD_L
DONE: ; Result in AL
RET
FIND_GCD ENDP

```

18 : Sum of numbers in ASCII

```

ORG 100h

.DATA
MSG1 DB 'Input 5 digits: $'
MSG2 DB 0DH, 0AH, 'Sum: $'
SUM DB 0

```

```

.CODE
MAIN PROC
MOV AX, @DATA
MOV DS, AX

; Display Input Prompt
MOV AH, 09h
LEA DX, MSG1
INT 21h

; Initialize Loop

```

```

MOV CX, 5
XOR BX, BX ; Use BL to hold the
running total

```

```

INPUT_LOOP:
MOV AH, 01h ; Function to read
character
INT 21h
SUB AL, 48 ; Convert ASCII to
actual number
ADD BL, AL ; Add value to total in
BL
LOOP INPUT_LOOP

```

```

; Display Output Message
MOV AH, 09h
LEA DX, MSG2
INT 21h

```

```

; --- HEX CONVERSION LOGIC ---
; BL now contains the sum (e.g.,
1+2+3+4+5 = 15 or 0Fh)

```

```

CMP BL, 10
JL IS_DIGIT ; Jump if result is 0-9

```

```

; If result is 10-15 (A-F)
ADD BL, 55 ; 10 + 55 = 65 ('A')
JMP DISPLAY

```

```

IS_DIGIT:
ADD BL, 48 ; 0 + 48 = 48 ('0')

```

```

DISPLAY:
MOV DL, BL
MOV AH, 02h ; Display single
character
INT 21h

```

```

; Return to DOS
MOV AH, 4CH
INT 21H
MAIN ENDP
END MAIN

```

19: Reverse and loop

```

ORG 0100H
.DATA
; Define the string with Carriage Return
(0Dh) and Line Feed (0Ah)
STR1 DB 'Islamic University of
Technology', '$'
REV_LEN DW 32 ; Length of the
string (33 chars - 1 for $)
NEWLINE DB 0DH, 0AH, '$'

```

```

.CODE
MAIN PROC
MOV AX, @DATA
MOV DS, AX

```

```

MOV CX, 10 ; Total 10 iterations
(20 prints total)

```

```

MASTER_LOOP:
PUSH CX ; Save outer loop
counter

```

```

; 1. Print Forward
CALL PRINT_FORWARD
CALL PRINT_NEWLINE

```

```

; 2. Print Reverse
CALL PRINT_REVERSE
CALL PRINT_NEWLINE

```

```

POP CX ; Restore outer loop
counter
LOOP MASTER_LOOP

```

```

MOV AH, 4CH ; Return to DOS
INT 21H
MAIN ENDP

```

```

; --- Procedure: Print Forward ---
PRINT_FORWARD PROC
MOV AH, 09H
LEA DX, STR1
INT 21H
RET
PRINT_FORWARD ENDP

```

```

; --- Procedure: Print Reverse ---
PRINT_REVERSE PROC
MOV CX, REV_LEN ; Number of
characters to print
MOV SI, OFFSET STR1
ADD SI, REV_LEN ; Point to the
last character before '$'
DEC SI ; Adjust pointer to
actual last char

```

```

BACKWARD_LOOP:
MOV DL, [SI] ; Get character at SI
MOV AH, 02H ; Display character
function
INT 21H
DEC SI ; Move pointer to
previous character
LOOP BACKWARD_LOOP
RET
PRINT_REVERSE ENDP

```

```

; --- Procedure: New Line ---
PRINT_NEWLINE PROC
MOV AH, 09H
LEA DX, NEWLINE
INT 21H
RET
PRINT_NEWLINE ENDP

```

```

END MAIN

```

Task 21: ASCII Range Forward and Reverse Logic: Digits (48-57), Upper (65-90), Lower (97-122).

```
MOV CX, 10
MOV DL, 48
FORW: MOV AH, 2 | INT 21h | INC DL |
LOOP FORW
```

Task 12 : Prime Digits Ascending

```
ORG 0100h
.DATA
    ARRAY      DB 8 DUP(?)
    PRIME_ARRAY DB 8 DUP(?)
    COUNT      DB 0          ; To keep
track of how many primes found
    PROMPT     DB 'Input 8 digits: $'
    MSG_OUT    DB 0DH, 0AH, 'Primes
Digits Ascending: $'

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX ; 1. Display Prompt
    MOV AH, 09h
    LEA DX, PROMPT
    INT 21h ; 2. Input 8 digits
    MOV CX, 8
    MOV SI, 0
INPUT_LOOP:
    MOV AH, 01h
    INT 21h
    SUB AL, 48 ; Convert ASCII to Decimal
    MOV ARRAY[SI], AL
    INC SI
    LOOP INPUT_LOOP ; 3. Filter Prime
Digits (2, 3, 5, 7)
    MOV CX, 8
    MOV SI, 0
    MOV DI, 0 ; Index for PRIME_ARRAY
FILTER_LOOP:
    MOV AL, ARRAY[SI]; Check against
prime digits
    CMP AL, 2
    JE IS_PRIME
    CMP AL, 3
    JE IS_PRIME
    CMP AL, 5
    JE IS_PRIME
    CMP AL, 7
    JE IS_PRIME
    JMP NEXT_DIGIT

IS_PRIME:
    MOV PRIME_ARRAY[DI], AL
    INC DI
    INC COUNT; Count primes we added
```

```
NEXT_DIGIT:
    INC SI
    LOOP FILTER_LOOP ; 4. Sort
PRIME_ARRAY (Bubble Sort)
    ; If no primes found, skip sorting
    CMP COUNT, 0
    JE DISPLAY_RESULTS

    XOR CX, CX
    MOV CL, COUNT; Outer loop
    DEC CL          ; Outer loop n-1
    JZ DISPLAY_RESULTS ; If only 1
prime, no need to sort

OUTER_SORT:
    PUSH CX
    LEA SI, PRIME_ARRAY
INNER_SORT:
    MOV AL, [SI]
    CMP AL, [SI+1]
    JLE NO_SWAP; For Ascending, skip if
Left <= Right
    XCHG AL, [SI+1]
    MOV [SI], AL
NO_SWAP:
    INC SI
    LOOP INNER_SORT
    POP CX
    LOOP OUTER_SORT

DISPLAY_RESULTS:
    ; 5. Display Output Message
    MOV AH, 09h
    LEA DX, MSG_OUT
    INT 21h; 6. Print the sorted
    XOR CX, CX
    MOV CL, COUNT
    CMP CL, 0
    JE FINISH
    MOV SI, 0
PRINT_LOOP:
    MOV DL, PRIME_ARRAY[SI]
    ADD DL, 48          ; Convert back to
ASCII
    MOV AH, 02h
    INT 21h

    MOV DL, ' '          ; Print space
    INT 21h

    INC SI
    LOOP PRINT_LOOP

FINISH:
    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

sum of a squared series

```
ORG 100h

.DATA
    A DB 1, 2, 3, 4          ; Array of numbers to
square (1, 4, 9, 16)
    RESULT DB 0
    MSG DB 'Sum of Squares: $'

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    XOR AX, AX          ; Clear AX
    MOV CX, 4          ; Array size is 4
    MOV SI, OFFSET A    ; Pointer to start
of array
    XOR BX, BX          ; BL will store our
final sum

SQUARE_LOOP:
    MOV AL, [SI]          ; Load number from
array
    MUL AL          ; AX = AL * AL
(Square the number)
    ADD BL, AL          ; Add the squared
result to BL
    INC SI          ; Move to next element
in array
    LOOP SQUARE_LOOP ; Repeat until
CX is 0

    ; Display Message
    MOV AH, 09h
    LEA DX, MSG
    INT 21h

    ; Display Result (Assuming sum < 10
for simplicity)
    ; For sums > 9, hex conversion or
multi-digit logic is needed
    MOV AL, BL
    ADD AL, 48          ; Convert to ASCII
    MOV DL, AL
    MOV AH, 02h
    INT 21h

    MOV AH, 4CH          ; Return to DOS
    INT 21H
MAIN ENDP
END MAIN
```

0. Write an assembly language program that will take an alphabet as an input (A~Z) or (a~z) and also take a given value N as input (0 to 9). Now, the output will show/display the N number character from the alphabet order starting from starting input alphabet (in the same case upper/lower as the given input).

Sample Input / Output:

Input Alphabet: B

Given Value N: 3

Output: E

```
org 100h
.data
character db "Enter the character: $"
number db "Enter the number: $"
counter db 0
.code
mov ax, @data
mov ds, ax
lea dx, character ; showing message
mov ah, 09h
int 21h

mov ah, 01 ;taking character input
int 21h
mov bl, al ; storing character in bl
; go to a new line with carriage return
MOV AH, 2
MOV DL, 0DH
INT 21h
MOV DL, 0AH
INT 21h
lea dx, number ; showing message
mov ah, 09h
int 21h
mov ah, 01 ; taking number input
int 21h

mov bh, al ; storing the number in bh
sub bh, 48 ; considering ascii value
add bh, bl ; adding values to get output character
; go to a new line with carriage return
MOV AH, 2
MOV DL, 0DH
INT 21h
MOV DL, 0AH
INT 21h

mov ah, 02 ; printing the output
mov dl, bh
int 21h
ret
```

Write an assembly language program that will accept an input string of 5 (five) letters in

LOWERCASE from the keyboard and displays the string in reverse order in UPPERCASE in a new line.

Sample Input / Output:

Input: abcdef

Output: FEDCBA

```
.data
lowercase db 5 dup(0)
uppercase db 5 dup(0)
.code
main proc
mov ax, @data
mov cl, 5
mov si, 0
loop1:
mov ah, 01h
int 21h
mov lowercase[si], al

add si, 1
loop loop1
mov cl, 5
mov si, 0
loop2:
mov di, cx
sub di, 1
mov al, lowercase[si]
mov uppercase[di], al
add si, 1
loop loop2
mov cx, 5
mov si, 0
loop3:
xor ax, ax
xor dx, dx
mov ah, 02h
mov dl, uppercase[si]
sub dl, 32
int 21h
add si, 1
loop loop3
ret
main endp
```

Write an assembly language program that will accept an input string of 5 (five) letters in UPPERCASE from the keyboard and displays the string in reverse order in LOWERCASE in a new line.

Sample Input / Output:

Input: ABCDEF

Output: fedcba

```
.model small
.stack 100h
.data

.code
main proc
include 'emu8086.inc'
```

printn "enter string size& Input: "

MOV ah, 01h

INT 21h

printn

MOV cl, al

SUB cl, 48

MOV bl, 0

MOV ah, 01h

input:

INT 21h

MOV [bx], al

INC bl

CMP bl, cl

JL input

finish:

printn

printn "OUTPUT : "

MOV ah, 02h

MOV bl, cl

DEC bl

rev:

MOV dx, [bx]

ADD dx, 32

INT 21h

DEC bl

CMP bl, 0

JGE rev

MOV ax, 4c00h

INT 21h

RET

MAIN ENDP

END MAIN

Write an assembly language program that will accept an input digit N (0 to 9) from the keyboard and finds the even digits up to that input and displays those in new lines.

Sample Input / Output:

Input: 9

Output: Even Digits: 0 2 4 6 8

```
org 0100h
.data
n db ?
arr db ?
print dw 'Even digits : ','$'
.code
main proc
mov ax, @data
mov ds, ax
mov si, offset print
;taking input from user
mov ah, 01h
int 21h
sub al, 48
mov n, al
mov cl, al
```

```

xor cl,cl
mov cl,n
mov bl,0

mov dl,0dh
mov ah,02h
int 21h
mov dl,0ah

int 21h
mov ah,09h
lea dx,print
int 21h

even:
mov dl,0h
mov ah,02h
int 21h

mov dl,bl
add dl,48
mov ah,02h
int 21h
add bl,2
cmp bl,cl
jle even

main endp
end main
ret

```

Write an assembly language program that will accept an input digit N (0 to 9) from the keyboard and finds the odd digits up to that input and displays those in new lines.

Sample Input / Output:

Input: 9

Output: Odd Digits: 1 3 5 7 9

```

ORG 0100h
.DATA
MSG DB 'Odd Digits : $'
COUNT DB ?

```

```

.CODE
MAIN PROC
MOV AX, @DATA
MOV DS, AX
MOV AH, 1 ; TAKING THE INPUT
INT 21H
SUB AL, 48 ; CONVERTING ASCII TO INTEGER
MOV COUNT, AL ; STORING AL INTO COUNT VARIABLE
MOV AH, 2
MOV DL, 0AH ; PRINTING NEWLINE
INT 21H
MOV DL, 0DH ; PRINTING CARRIAGE
RETURN
INT 21H

```

```

MOV AH, 9 ; PRINTING MSG
MOV DX, OFFSET MSG
INT 21H
MOV AH, 2 ; STARTED PRINTING
OUTPUT
MOV BL, 1
LOOP0:
CMP BL, COUNT
JG EXIT
MOV DL, BL
ADD DL, 48
INT 21H
MOV DL, ''
INT 21H
ADD BL, 2
JMP LOOP0
EXIT:
RET
MAIN ENDP

```

Write an assembly language program that will accept an input of 5 (five) digits (0 to 9)

from the keyboard randomly and rearrange them in ascending order.

Sample Input / Output:

Input: 2 4 5 3 2

Output: Ascending: 2 2 3 4 5

```

include 'emu8086.inc'
org 100h
.data
arr db 5 dup(?)
input_array db 'Input:$'
output_array db 'Ascending:$'
.code
main proc
mov ah,9
lea dx,input_array ; output of the input
string
int 21h
mov cx,5 ; takes the value 5 for the input
loop
lea bx, arr; load effective address of arr
taken in bx
mov ah,1
input:
int 21h
mov [bx],al; input taken into the arr array
inc bx
loop input

```

```

mov cx,5
dec cx
outerloop:
mov bx, cx
mov si,0
comploop:
mov al,arr[si]
mov dl,arr[si+1]
cmp al,dl
jc noSwap

```

```

mov arr[si],dl
mov arr[si+1],al
noSwap:
inc si
dec bx
jnz comploop
loop outerloop
mov ah,2
mov dl,10
int 21h
mov dl,13
int 21h

```

```

mov ah,9
lea dx,output_array
int 21h
mov cx,5
mov bx, offset arr
Outputs:
mov dl,[bx]
mov ah,2
int 21h
inc bx
loop Outputs
main endp
ret

```

Write an assembly language program that will accept an input of 5 (five) digits (0 to 9) from the keyboard randomly and sort-out the odd and even digits from them.

Sample Input / Output:

Input: 2 4 5 3 2

Output: Odd: 3 5

Even: 2 2 4

ORG 0100H

.DATA

```

input DB 'Input: $' ; define string to display
odd DB 0DH, 0AH,'Odd: $'
even DB 0DH, 0AH,'Even: $'
str DB 5 dup(0)

```

```

.CODE
MAIN PROC
mov ax, @DATA
mov ds, ax
xor ax,ax
mov si, OFFSET input
mov ah, 09h
int 21h
mov cl,5
mov si,0
mov ah, 01h
takeinput: int 21h
sub al, 48
mov str[si], al
inc si
loop takeinput

```

```

mov ah, 09h
lea dx,odd
int 21h

mov cl,5
mov si,0
l1: xor ax,ax
call oddnum ; call procedure
inc si
loop l1

mov ah, 09h
lea dx,even
int 21h

mov cl,5
mov si,0

l3: xor ax,ax
call evennum ; call procedure
inc si
loop l3
mov ah, 4Ch
mov al, 00h ; a code after procedure call
and return
int 21h ; exit to DOS
MAIN ENDP
oddnum proc ; declare a procedure
named oddnum
mov al,str[si]
mov bl,2
div bl
cmp ah, 0
jz exit
mov ah, 02h
mov dl, str[si]
add dl,48
int 21h
ret
exit: ret
oddnum ENDP ; end of procedure
evennum proc ; declare a procedure
named evennum
mov al,str[si]
mov bl,2
div bl
cmp ah, 0
jnz exit2
mov ah, 02h
mov dl, str[si]
add dl,48
int 21h
ret
exit2: ret
evennum ENDP ; end of procedure
evennum

END MAIN ; end of program

```

Write an assembly language program that will accept an input of 8 (eight) digits (0 to 9) from the keyboard randomly and sort-out the odd digits in ascending order.

Sample Input / Output:

Input: 2 4 7 5 6 1 8 3

Output: Odd Ascending: 1 3 5 7

; You may customize this and other

start-up templates;

; The location of this template is

c:\emu8086\inc\0_com_template.txt

```

org 100h
.model small
.data
arr db 8 dup(?)
odd_arr db 8 dup (?)
output_message db 'Output: Odd
Ascending:$'
input db 'Input:$'
.code
main proc
mov ax, @data
mov ds,ax
mov cx,8
mov bx,offset arr
mov dx,offset input
mov ah,09h
int 21h
xor dx,dx
mov ah,1
inputs:
int 21h

```

```

mov [bx],al
inc bx
Loop inputs

```

```

mov cx,8
dec cx
Outerloop:
mov bx,cx
mov si,0
Comploop:
mov al,arr[si]
mov dl,arr[si+1]
cmp al,dl
jc noSwap
mov arr[si],dl
mov arr[si+1],al
noSwap:
inc si
dec bx
jnz Comploop
loop Outerloop
mov ah,2
mov dl,10
int 21h
mov dl,13
int 21h

```

```

mov dx,offset output_message
mov ah,09h
int 21h
xor si,si
xor dx,dx

```

```

mov cx,8

```

```

mov bx,offset arr
xor dx,dx
mov si,9
xor ax,ax

```

```

Outputs:
dec si
xor ax,ax
mov cx,si
mov al,[bx]
sub al,48
mov cl,02h
div cl
cmp ah,0
je continue
mov dl,[bx]
mov ah,2
int 21h
mov dl,' '
mov ah,2
int 21h
continue:
inc bx
mov cx,si
loop Outputs
main endp
ret

```

Write an assembly language program that will accept an input of 8 (eight) digits (0 to 9) from the keyboard randomly and sort-out the odd digits in descending order.

Sample Input / Output:

Input: 2 4 7 5 6 1 8 3

Output: Odd Descending: 7 5 3 1

```

INCLUDE 'emu8086.inc'
org 100h

```

```

.data
arr db 8 dup(?)
.code
main proc
mov ax, @DATA
mov ds, ax
mov cx, 8
lea bx, arr
mov ah, 1
; Taking inputs
inputs:
int 21h
mov [bx], al

```

```

inc bx
loop inputs

mov cx, 8
dec cx
traverse:
mov bx, cx
mov si, 0
compare:
mov al, arr[si]
mov dl, arr[si+1]
cmp dl, al
jc no
mov arr[si], dl
mov arr[si+1], al
no:
inc si
dec bx
jnz compare
loop traverse

mov ah, 2
mov dl, 10
int 21h
mov dl, 13
int 21h
;Printing the sorted odds
mov cx, 8
lea si, arr
PRINT "Odd Descending: "
Outputs:
mov al, [si]
test al, 1
jz no_print
mov dl, [si]
mov ah, 2
int 21h
mov dl, 32
mov ah, 2
int 21h
no_print:
inc si
loop Outputs
main endp
ret

```

Write an assembly language program that will accept an input of 8 (eight) digits (0 to 9) from the keyboard randomly and sort-out the even digits in ascending order.

Sample Input / Output:
Input: 2 4 7 5 6 1 8 3
Output: Even Ascending: 2 4 6 8
INCLUDE 'emu8086.inc'
ORG 100H

```

.DATA
array DB 8 DUP(?)
.CODE

```

```

MAIN PROC
PRINT "Enter 8 digits: "
MOV AH, 02H
MOV DL, 0DH
INT 21H
MOV DL, 0AH
INT 21H
MOV SI, OFFSET array
MOV CX, 8
MOV BX, 0H
LOOP1:
MOV AH, 01H
INT 21H
MOV [SI], AL
MOV DL, 32
MOV AH, 02H
INT 21H
INC SI
LOOP LOOP1
MOV AH, 02H
MOV DL, 0DH
INT 21H
MOV DL, 0AH
INT 21H
MOV CX, 7
LOOP2:
MOV BX, CX
MOV SI, 0
LOOP3:
MOV AL, array[SI]
MOV DL, array[SI+1]
CMP AL, DL
JC NoSWAP
MOV array[SI], DL

MOV array[SI+1], AL
NoSWAP:
INC SI
DEC BX
JNZ LOOP3
LOOP LOOP2
MOV CX, 8
MOV SI, OFFSET array
PRINT "Even Ascending: "
LOOP4:
MOV AL, [SI]
TEST AL, 1
JNZ EVEN
MOV DL, [SI]
MOV AH, 02H
INT 21H
MOV DL, 32
MOV AH, 02H
INT 21H
EVEN:
INC SI
LOOP LOOP4

MAIN ENDP
RET

```

Write an assembly language program that will accept an input of 8 (eight) digits (0 to 9) from the keyboard randomly and sort-out the even digits in descending order.

Sample Input / Output:
Input: 2 4 7 5 6 1 8 3
Output: Even Descending: 8 6 4 2
include 'emu8086.inc'
org 100h

```

.data
arr db 8 dup(?)
even_checker db 2
.code
main proc

print "Enter 8 number in array:"
mov cx,8
mov bx,offset arr
mov ah,1
inputs:
int 21h
mov [bx],al
inc bx
loop inputs

mov cx,8
dec cx
outerloop:
mov bx, cx
mov si,0
comploop:
mov al,arr[si]
mov dl,arr[si+1]
cmp dl,al
jc noSwap
mov arr[si],dl
mov arr[si+1],al

```

```

noSwap:

inc si
dec bx
jnz comploop
loop outerloop
mov ah,2
mov dl,10
int 21h
mov dl,13
int 21h
print "After Sorting Array: "
mov cx,8
mov si, offset arr

```

```

Outputs:
mov al,[si]
test al,1
jnz even

```

```

mov dl,[si]
mov ah,2
int 21h
mov dl,32
mov ah,2
int 21h
even:
inc si
loop Outputs
main endp
ret

```

Write an assembly language program that will accept an input of 8 (eight) digits (0 to 9) from the keyboard randomly and finds the prime digits in descending order. Sample Input / Output:

Input: 2 4 7 5 6 1 8 3
Output: Primes Digits Descending: 7 5 3 2

```

ORG 0100h
.DATA
input DB 'Input: $'
outputp DB 'Prime Digits Descending: $'
arr DB 8 dup(0)
.CODE
MAIN PROC
mov ax,@DATA
mov ds,ax
xor ax,ax
mov dx,OFFSET input
mov ah,09h
int 21h
mov cx,8
mov si,0
mov ah,01h
l1: int 21h
sub al,48
mov arr[si],al
inc si
loop l1
mov ah,2
mov dl,0Ah
int 21h
mov dl,0Dh
int 21h
mov cx,8
dec cx
continue:
mov bx,cx
mov si,0
compare:
mov al,arr[si]
mov dl,arr[si+1]
cmp al,dl
jg unchanged
mov arr[si],dl
mov arr[si+1],al
unchanged:
inc si

```

```

dec bx

jnz compare
loop continue
mov bx,offset arr
mov ah,09h
lea dx,outputp
int 21h
mov cl,8
mov si,0
l2: xor ax,ax
call prime
inc si
loop l2
MOV AH,4Ch
MOV AL,00h
INT 21h
MAIN ENDP
prime proc
cmp arr[si],0
jz exit
cmp arr[si],1
jz exit
cmp arr[si],2
jz exit2
cmp arr[si],3
jz exit2
cmp arr[si],4
jz exit
cmp arr[si],5
jz exit2
cmp arr[si],6
jz exit
cmp arr[si],7
jz exit2
cmp arr[si],8
jz exit
cmp arr[si],9
jz exit
exit2: mov ah,02h
mov dl,arr[si]
add dl,48
int 21h
ret
exit: ret
prime ENDP

```

END MAIN
Write an assembly language program that will accept an input of 8 (eight) digits (0 to 9) from the keyboard randomly and sort-out the odd prime digits in ascending order. Sample Input / Output:
Input: 2 4 7 5 6 1 8 3
Output: Odd Primes Ascending: 3 5 7 ; Done by Nahian_180041136
org 0100h
.data

```

InputMsg DB 'Input: $' ; Showing Input
message => 'Input: '
OutputMsg DB 'Output: Odd Primes
Ascending: $' ; Showing Output message
=> 'Output: Odd
Primes Ascending: '
OddPrimeMsg DB 'Prime: $' ; Showing
Odd Prime message => 'Prime: '
OddPrimeArr DB 8 DUP(0,20h),'$' ;
Initializing array for prime digits

```

;20h is inserted in array for space

```

count_3 db 0
count_5 db 0
count_7 db 0

```

```

.code
main proc
lea dx, InputMsg ; set DX to point to 1st
element of string array InputMsg
mov ah, 09h
int 21h
xor cl, cl ; clearing for counter
mov cl, 8 ; intializing counter value 5 to
take 5 inputs
Loop_1:
mov ah, 01h ; taking singe-key input
int 21h
; only 3, 5, 7 are odd prime from 0~9
cmp al, 33h
jz inc_3_call

cmp al, 35h
jz inc_5_call
cmp al, 37h
jz inc_7_call

```

```

jmp NextLoop
inc_3_call:
call inc_3 ; calling inc_3 proceure
procedure to assign prime digits
jmp NextLoop ; skipping other procedure
call
inc_5_call:
call inc_5
jmp NextLoop
inc_7_call:
call inc_7
jmp NextLoop
NextLoop:
mov ah, 2 ; display character function
mov dl, ' ' ; character is a space
int 21h
loop Loop_1
mov ah, 2
mov dl, 0DH ; carriage return (start of a
line)
int 21h
mov dl, 0AH ; line feed (new line)
int 21h

```

```

lea dx, OutputMsg ; set DX to point to 1st
element of string array OutputMsg
mov ah, 09h
int 21h
xor cx, cx
cmp count_3, 0
jz work_with_5
mov cl, count_3
Loop_print_3:

mov ah, 2 ; display character function
mov dl, '3' ; character is a space
int 21h
mov ah, 2 ; display character function
mov dl, ' ' ; character is a space
int 21h
loop Loop_print_3
work_with_5:
cmp count_5, 0
jz work_with_7
mov cl, count_5
Loop_print_5:
mov ah, 2 ; display character function
mov dl, '5' ; character is a space
int 21h
mov ah, 2 ; display character function
mov dl, ' ' ; character is a space
int 21h
loop Loop_print_5
work_with_7:
cmp count_7, 0
jz work_done
mov cl, count_7
Loop_print_7:
mov ah, 2 ; display character function
mov dl, '7' ; character is a space
int 21h
mov ah, 2 ; display character function
mov dl, ' ' ; character is a space
int 21h
loop Loop_print_7
work_done:
mov ah, 4ch
mov al, 00h ; a code after procedure call
and return
int 21h
main endp
inc_3 proc

inc count_3 ; if input digit is equal to 3,
increasing count_3 counter value
ret
inc_3 endp
inc_5 proc
inc count_5 ; if input digit is equal to 5,
increasing count_5 counter value
ret
inc_5 endp
inc_7 proc
inc count_7 ; if input digit is equal to 7,
increasing count_7 counter value

```

```

ret
inc_7 endp
Write an assembly language program
that will accept an input of 8 (eight)
digits (0 to 9)
from the keyboard randomly and sorts
them in ascending order.
Sample Input / Output:
Input: 2 4 3 5 6 1 8 3
Output: 1 2 3 3 4 5 6 8
include 'emu8086.inc'
org 100h
.data
arr db 5 dup(?)
.code
main proc
print "Enter 5 number in array:"
mov cx, 5
mov bx, offset arr
mov ah, 1
inputs:
int 21h
mov [bx], al
inc bx
loop inputs

mov cx, 5
dec cx

outerloop:
mov bx, cx
mov si, 0
compploop:
mov al, arr[si]
mov dl, arr[si+1]
cmp al, dl
jc noSwap
mov arr[si], dl
mov arr[si+1], al
noSwap:
inc si
dec bx
jnz comploop
loop outerloop
mov ah, 2
mov dl, 10
int 21h
mov dl, 13
int 21h
print "After Sorting Array: "
mov cx, 5
mov bx, offset arr
Outputs:
mov dl, [bx]
mov ah, 2
int 21h
mov dl, 32
mov ah, 2
int 21h
inc bx
loop Outputs

```

```

main endp
ret

```

```

Write an assembly language program
that will accept an input of 8 (eight)
digits (0 to 9)
from the keyboard randomly and sorts
them in descending order.
Sample Input / Output:
Input: 2 4 3 5 6 1 8 3
Output: 8 6 5 4 3 3 2 1
include 'emu8086.inc'
org 100h
.model small
.data
arr db 8 dup(?)
.code
main proc
mov ax, @data
mov ds, ax
print "Input:"
mov cx, 8
mov bx, offset arr
mov ah, 1
inputs:
int 21h
mov [bx], al
inc bx
Loop inputs
mov cx, 8
dec cx
OuterLoop:
mov bx, cx
mov si, 0
CompLoop:
mov al, arr[si]

mov dl, arr[si+1]
cmp al, dl
jnc noSwap
mov arr[si], dl
mov arr[si+1], al
noSwap:
inc si
dec bx
jnz CompLoop
loop OuterLoop

mov ah, 2
mov dl, 10
int 21h
mov dl, 13
int 21h
print "Output:"
mov cx, 8
mov bx, offset arr
Outputs:
mov dl, [bx]
mov ah, 2
int 21h

```



```
inc bx
loop Outputs
main endp
ret
Write an assembly language program that will accept an input of 5 (five) digits (0 to 9) from the keyboard and finds the summation of the digits and displays the result in HEX digit.
Sample Input / Output:
Input: 1 2 3 4 5
```

Output: Sum: F
ORG 100h

```
.DATA ; Data segment starts
sum DB 0
flag DB 33 DUP(0)
hexdigit1 DB ?
hexdigit2 DB ?
inputMessage DB 'Enter 5 decimal digits: $'
outputMessage DB 'Hex Sum: $'
.CODE ; Code segment starts
MAIN PROC
MOV AX, @DATA
MOV DS, AX
XOR AX, AX
```

; Print Message to enter values

```
MOV DX, OFFSET inputMessage
MOV AH, 09h
INT 21h
CALL TAKE_INPUT
CALL CALC_SUM
CALL DISPLAY_OUTPUT
```

```
RET
MAIN ENDP
```

```
TAKE_INPUT PROC
XOR CX, CX ; Clear count register
MOV CL, 5 ; To take 5 inputs
MOV SI, 0 ; Initialize SI
input_digits:
MOV AH, 01h
```

```
INT 21h
SUB AL, 48 ; To get decimal value of the inputs
ADD sum, AL
INC SI
LOOP input_digits
RET
TAKE_INPUT ENDP
```

```
CALC_SUM PROC
CMP sum, 32
```

```
JL less32
MOV flag[32], 1
SUB sum, 32
less32:
CMP sum, 16
JL less16
MOV flag[16], 1
SUB sum, 16
less16:
CMP sum, 8
JL less8
MOV flag[8], 1
SUB sum, 8
less8:
CMP sum, 4
JL less4
MOV flag[4], 1
SUB sum, 4
less4:
CMP sum, 2
JL less2
MOV flag[2], 1
SUB sum, 2
less2:
CMP sum, 1
JL less1
MOV flag[1], 1
SUB sum, 1
less1:
XOR AX, AX
MOV AL, 2
MUL flag[32]
ADD hexdigit1, AL
MOV AL, 1
MUL flag[16]
ADD hexdigit1, AL
MOV AL, 8
MUL flag[8]
ADD hexdigit2, AL
MOV AL, 4
MUL flag[4]
ADD hexdigit2, AL
MOV AL, 2
MUL flag[2]
ADD hexdigit2, AL
MOV AL, 1
MUL flag[1]
ADD hexdigit2, AL
CMP hexdigit1, 10
JL notChar1
ADD hexdigit1, 7
notChar1:
CMP hexdigit2, 10
JL notChar2
ADD hexdigit2, 7
notChar2:

RET
CALC_SUM ENDP
```

```
DISPLAY_OUTPUT PROC
MOV AH, 02h
MOV DL, 0Ah ; to print new line
INT 21h
MOV DL, 0Dh ; carriage return
INT 21h
MOV DX, OFFSET outputMessage
MOV AH, 09h
INT 21h
XOR DX, DX
XOR AX, AX
MOV AH, 02h
MOV DL, hexdigit1
CMP DL, 0
JE dontPrint:
ADD DL, 48
INT 21h
dontPrint:
MOV DL, hexdigit2
ADD DL, 48
INT 21h
```

```
RET
DISPLAY_OUTPUT ENDP
Write an assembly language program that will display an array inputted string “Islamic University of Technology” and it’s reverse string one after another for 10 times each (forward and reverse print) in total 20 (twenty times) in different lines with line feed..
ORG 0100h
.DATA
Forward DW 'ISLAMIC UNIVERSITY OF TECHNOLOGY $'
```

```
Reverse DW 'YGOLONHCET FO YTISREVINU CIMALSI $'
```

```
.CODE
MAIN PROC
MOV CX, 10 ;each string for 10 times
Level:
MOV AH, 9
MOV DX, OFFSET Forward ; for geetting the forward string
INT 21H
MOV AH, 2
MOV DL, 0DH
INT 21h
MOV DL, 0AH
INT 21h
MOV AH, 9
MOV DX, OFFSET Reverse ; then for getting the reverse string next
INT 21H
MOV AH, 2
MOV DL, 0DH
INT 21h
```

```
MOV DL, 0AH
INT 21h
```

LOOP Level ; for repeating the 2 strings again in the same order

```
MAIN ENDP
END MAIN
RET
END MAIN
```

Write an assembly language program that will display all the ASCII characters at REVERSE ORDER.

```
.DATA
Message DB '128 ASCII char in reverse
order:',13,10,'$'
.CODE
MAIN PROC
MOV AX, @DATA
MOV DS, AX
LEA DX, Message
MOV AH, 9
INT 21H
MOV CX, 128 ; initialize CX
MOV AH, 2
MOV DL, 127 ; initialize DL with last
ASCII character
top: ; loop label
INT 21H ; print ASCII character
DEC DL ; decrement DL to next ASCII
character
DEC CX ; decrement CX
JNZ top ; jump to label @LOOP if CX is 0
MOV AH, 4CH
INT 21H
MAIN ENDP
END MAIN
```

Write an assembly language program that will display only the ASCII characters of all DIGITS, ALPHABETS (upper and lower) at FORWARD and REVERSE ORDER (in different lines).

```
.MODEL SMALL
.STACK 100H
.DATA
PROMPT DB 'The ASCII values are : $'
.CODE
MAIN PROC

MOV AX, @DATA
MOV DS, AX
LEA DX, PROMPT
MOV AH, 9
INT 21H
MOV CX, 10
MOV AH, 2
MOV DL, 48
```

```
myLOOP_1:
INT 21H
INC DL
DEC CX
JNZ myLoop_1
```

```
MOV CX, 26
MOV AH, 2
MOV DL, 65
```

```
myLOOP_2:
INT 21H
INC DL
DEC CX
JNZ myLoop_2
```

```
MOV CX, 26
MOV AH, 2
MOV DL, 97
```

```
myLOOP_3:
INT 21H

INC DL
DEC CX
JNZ myLoop_3
```

```
MOV CX, 10
MOV AH, 2
MOV DL, 57
```

```
myLOOP_4:
INT 21H
DEC DL
DEC CX
JNZ myLoop_4
```

```
MOV CX, 26
MOV AH, 2
MOV DL, 90
```

```
myLOOP_5:
INT 21H
DEC DL
DEC CX
JNZ myLoop_5
```

```
MOV CX, 26
MOV AH, 2
MOV DL, 122
```

```
myLOOP_6:
INT 21H
DEC DL
```

```
DEC CX
JNZ myLoop_6
```

```
MOV AH, 4CH
INT 21H
MAIN ENDP
```

END MAIN

Write an assembly language program that will accept an input of 5 (five) digits (0 to 9) from the keyboard randomly and rearrange them in descending order.

Sample Input / Output:

Input: 2 4 5 3 2

Output: Descending: 5 4 3 2 2 ;180041128

```
org 100h
.DATA
input db 5 dup(0)
prompt db 10,13,'Enter 5 digits to be
sorted in DESCENDING order: $'
answer db 10,13,'Descending: $'
SPACE db ' $'
.CODE
MAIN PROC
mov ax, @DATA
mov ds, ax
xor ax, ax
mov ah, 09h ;Character string output
lea dx, prompt ;asking for 5 digits to be
sorted
int 21h
mov ah, 01h
mov cx, 5
mov si, OFFSET input ;input array
address
inloop:
int 21h
mov [si], al
inc si
Loop inLoop
```

```
Call Sort ;calling the sorting procedure.
mov ah, 09h
lea dx, answer ;outputting "Descending"
int 21h
mov cx, 5
mov si, OFFSET input
outloop:
mov ah, 02h ;for Character output
mov dx, [si]
int 21h
mov ah, 09h ;for String output to print
SPACE
lea dx, SPACE
int 21h
inc si
Loop outLoop
mov ah, 4ch ;return
mov ah, 00h
int 21h
MAIN ENDP
; the sort is a bubble sort algorithm
Sort PROC
mov cx, 5
dec cx ;The outer loop will spin 4 times as
```

after each loop, the last element of the array will be the smallest.

outerloop:
mov bx, cx ;bx will be used as the counter
for

the inner loop

mov si,0
compLoop: ;inner loop for comparison
mov al, input[si]
mov dl, input[si+1]
cmp al,dl
jnc noSwap ;if there is no carry in after

comparison, no swap will happen

mov input[si], dl

mov input[si+1], al
noSwap:
inc si
dec bx
jnz comploop ;the comparison loop will
repeat if bx is not

0

Loop outerLoop
ret ;return to main
Sort ENDP
END MAIN

**Write an assembly language program
that will accept an input of 2 (two)
digits (0 to 9)
from the keyboard randomly and finds
the Greatest Common Divisor of two
digits.**

Sample Input / Output:

Input: 2 3 Input: 2 4

Output: GCD: 1 Output: GCD: 2

ORG 0100H

.DATA

A DB ?

B DB ?

PROMPT_1 DB 'Input 2 Numbers: ', '\$'

PROMPT_2 DB 'GCD: ', '\$'

.CODE

MAIN PROC

LEA DX, PROMPT_1

MOV AH, 9

INT 21h

MOV AH, 1

INT 21H

MOV A, AL

MOV AH, 2

MOV DL, ''

INT 21H

MOV AH, 1

INT 21H

MOV B, AL

MOV AH, 2

MOV DL, ''

INT 21H

SUB A, 48

SUB B, 48

MOV AL, A

MOV BL, B

CMP AL, BL

JG GREATER

JMP LESS

GREATER:

XOR CX, CX

MOV CL, B

JMP EXIT

LESS:

XOR CX, CX

MOV CL, A

EXIT:

LOOP0:

XOR AX, AX

MOV AL, A

DIV CL

CMP AH, 0

JNE NO

MOV AL, B

DIV CL

CMP AH, 0

JNE NO

JMP PRINT

NO:

LOOP LOOP0

PRINT:

MOV AH, 2

mov dl,0Ah ;carriage return

int 21h

mov dl,0Dh ;new line

int 21h

LEA DX, PROMPT_2

MOV AH, 9

INT 21h

MOV AH, 2

MOV DL, CL

ADD DL, 48

INT 21H

RET

MAIN ENDP